Trust-Calibrated Multi-Agent Scientific Deliberation for Mitigating Sycophantic Consensus in LLM Reasoning

Submitted By

Md. Atikur Rahaman (0112310298)
Rakibul Hasan (0112310530)
Md. Salman Rohoman Nayeem (0112310484)
Pratay Paul (0112310163)
Yousuf Kamal Himel (0112310526)
Mst. Farjana Akter Limu (0112310535)

Supervision:
Mohammad Nurul Huda,PHD
Professor, Dept. of Computer Science and Engineering
United International University.

Submitted in partial fulfilment of the requirements
of the degree of Bachelor of Science in Computer Science and Engineering

2026

Department of Computer Science and Engineering
United International University

Abstract

Large language models answer many scientific questions well, yet they still hallucinate and follow broken lines of reasoning. Multi-agent debate is one answer to this. Several models argue the same question before a final answer is chosen. The weak point is the final decision. Most debate systems count votes, so the majority wins. A confident but wrong majority can drag a correct minority agent onto the wrong side, and the group settles on a wrong answer. That is the sycophantic consensus problem. No current method checks an agent's claims against outside scientific evidence before that agent gains influence. This work changes how the debate is settled. The aim is to weight each agent by evidence instead of headcount. Every response is split into small factual claims. For each claim, the system retrieves relevant scientific literature and checks whether the evidence supports it or contradicts it. Each agent carries a trust score that rises and falls as these checks build up across the rounds. Unsupported claims lose influence, and the score stays bounded so no agent dominates the discussion. The reward is a correct minority that survives. Its influence no longer depends on the number of voices behind it. The group becomes less likely to lock onto a wrong answer early. No fine-tuning is needed. The framework should reduce hallucinations because each surviving claim is tied to its source passage. For comparison, the study tests against majority voting and other baselines. The final answer is returned with its supporting citations. The evaluation reports four rates: answer accuracy, consensus-collapse, minority-preservation, and evidence-calibration.

Acknowledgements

This project was not work of six students alone. Many people helped, and we thank them sincerely.

Dr. Mohammad Nurul Huda supervised the work. He kept the scope realistic when we tried to add too much, and he pushed us to make the trust score and evaluation concrete. The report is stronger because of his comments.

Dr. Ohidujjaman taught the FYDP course. Her deadlines and templates gave us a clear structure. Her quick replies helped the team stay on track across three trimesters.

Classmates and friends helped in practical ways. They read drafts and pointed out unclear paragraphs. They also helped us debug LaTeX and vLLM setup issues late in the lab. Our families supported us throughout this time. Their patience during long nights of writing and their steady encouragement kept us going.

Publication List

The publications connected to this research are listed below. One journal article is currently under review, and further submissions are planned once the evaluation results are ready.

Journal Articles

1. Trust-Calibrated Multi-Agent Scientific Deliberation for Mitigating Sycophantic Consensus in LLM Reasoning (In preparation)

Conference Papers

1.

Additional Publications

Following is the list of relevant publications published in the course of the research that is not included in the thesis:

1.

Table of Contents

List of Figures

List of Tables

Chapter 1

Introduction

This chapter sets the scene for the whole report. Background and problem come first, followed by the motivation, the objectives, a short account of the methodology, the expected outcome, and the structure of the remaining chapters.

1.1 Project Overview

Large language models now handle many reasoning tasks, from scientific question answering to tutoring and decision support. A single model can still be confidently wrong. It may state an incorrect answer in fluent and assured language, which makes the mistake hard to catch. Multi-agent debate was proposed to reduce this problem. Several agents answer the same question and read each other's reasoning, then revise their positions across a few rounds. Only after that is a final answer selected [1]. Chain-of-thought prompting gives each agent a way to expose its intermediate steps [2], which makes the reasoning inside a debate easier to inspect. The hard part is the final step, where the separate answers are combined. Most debate systems settle disagreements by majority vote. Each agent receives the same weight, so the most common answer wins. This rule treats popularity as a stand-in for correctness. In scientific reasoning, a better answer should be backed by evidence rather than repeated by more agents. The study examines a trust-calibrated multi-agent deliberation framework. It adjusts each agent's influence during the debate based on how well its claims match retrieved external evidence. The final answer then depends on evidence-grounded trust rather than a raw headcount.

1.2 Motivation

Majority voting can fail in a way that is easy to miss. A debate may look like it reached agreement while it has quietly dropped a correct answer held by one agent. Recent studies of multi-agent debate report that agents often copy or switch answers under social pressure instead of reasoning on their own. The correct answer is present in the discussion but ignored in more than 20 percent of wrong-answer cases [3]. Work on model behaviour

points to a related tendency, where models trained to be agreeable grow more sycophantic and less accurate [4]. A confident majority pushes a correct minority agent to give up its answer, and majority voting then does more than pick the wrong result. It hides the failure behind an appearance of consensus. This matters most for scientific questions, where a wrong but confident consensus can mislead a user into trusting an answer that deserved a closer check. Each existing method attacks only part of the problem. iMAD decides when a debate is worth running at all [5]. DebUnc scales agent influence by self-reported confidence [6]. Mixture-of-Agents folds all outputs together through static aggregation [7]. None of these ties an agent's influence to whether its claims survive a check against external evidence while the debate is still running. That connection is exactly what this study adds.

1.3 Objectives

The main objective is to reduce sycophantic consensus in multi-agent LLM debate. The study replaces plain majority voting with trust that is grounded in evidence. To do this, the work first needs a clear picture of the failure. A confident but wrong majority can push a correct minority agent to change its answer [3, 8]. Next, it needs a bounded trust score. The score rises when external evidence supports an agent's claims and falls when the evidence contradicts them. The update happens before the final answer is fixed. The study also tests the method against majority voting and other debate and aggregation baselines [9, 10]. Performance is measured with four rates: answer accuracy, consensus-collapse, minority-preservation, and evidence-calibration.

1.4 Methodology

The study uses an experimental design. A baseline debate system is built first. Several heterogeneous LLM agents answer the same scientific question and revise their positions after reading each other's reasoning across rounds [1]. The baseline keeps majority voting. This makes the usual failure mode visible and lets it be measured [9]. Next, each agent's reasoning is split into smaller factual claims. These claims are checked against scientific evidence from external sources, such as academic papers and reference databases [11]. Contradictions between a claim and the retrieved evidence are detected and scored [12]. The idea that external tools can verify and correct model output supports this step [13]. Verdicts then feed into trust updates. This follows work that uses per-agent confidence to steer debate [6]. A supported claim raises the agent's trust, and a contradicted claim lowers it. Uncertain claims are handled in a conservative way so the score does not jump on weak evidence. The trust score stays bounded, so no agent gains unlimited influence or drops out completely. The final answer comes from trust-weighted aggregation, not a simple count. The framework is tested on scientific reasoning datasets such as GPQA and MMLU-Pro. Evaluation checks whether the mechanism lowers consensus collapse and whether correct minority answers survive. It also checks whether overall accuracy holds,

all without any fine-tuning. Figure 1.1 shows the end-to-end flow.

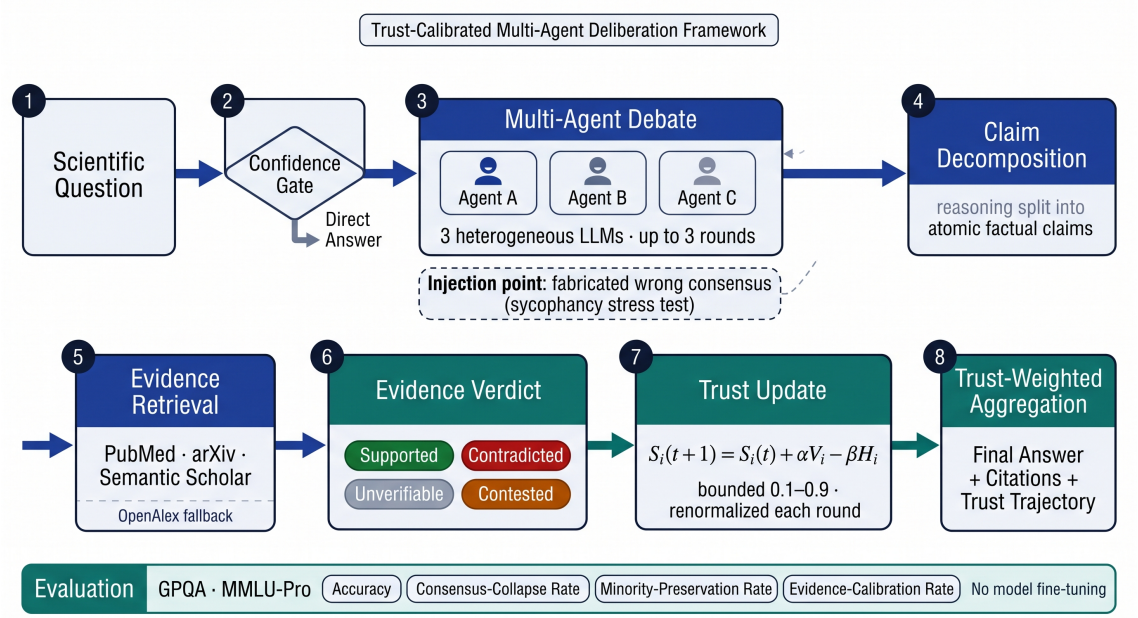

Figure 1.1: End-to-end flow of the trust-calibrated multi-agent deliberation framework: debate, claim-level evidence verification, per-round trust updates, and trust-weighted aggregation.

1.5 Project Outcome

The main outcome is a reproducible framework for trust-calibrated scientific deliberation with multiple agents. Agent influence is updated from retrieved evidence while the debate runs. The final answer then rests on evidence-weighted trust, not a majority vote. The study also produces an evaluation protocol. The protocol detects sycophantic consensus [3] and tracks how often correct minority answers survive [8]. It also measures the method against established baselines. Dynamic trust calibration should make debate more dependable on scientific questions. A wrong answer can still remain. The decision is still more transparent because the method records which agents the evidence supported and which it contradicted.

1.6 Organization of the Report

The rest of the report moves from background to results. Chapter 2 covers multi-agent debate and sycophancy. It also defines the gap this study addresses. The requirements, the methodology, the system design, and the project plan sit in Chapter 3.

Chapter 4 holds the implementation setup and the testing procedure. It also presents the results with discussion. Standards, constraints, sustainability, and the complex engineering problem analysis fill Chapter 5. Chapter 6 closes with a summary and limitations.

It also outlines directions for future work.

Chapter 2

Background

The chapter introduces the concepts needed to understand the system described in the report and reviews research on multi-agent debate, sycophancy, and agent influence. It then explains the gap addressed by the study.

2.1 Preliminaries

Large language models (LLMs) generate answers one token at a time, but their outputs can still contain unsupported or incorrect reasoning. Fluent wording does not guarantee that an answer is correct [14]. Chain-of-thought prompting asks a model to expose intermediate reasoning steps before giving a final answer, which makes the reasoning easier to inspect and gives later components more information to evaluate [2]. These steps are useful evidence about how an answer was formed, but they should not be treated as proof of correctness by themselves. Multi-agent debate takes this further. Several agents solve the same problem on their own, then see the other answers and explanations, discuss the differences, and revise their positions across a fixed number of rounds. Only after the discussion is a final answer chosen [1]. Independent attempts help because different agents expose different errors, and one agent can question another agent's reasoning. How the answers get combined still matters. Majority voting is the common choice: every agent carries the same weight and the most frequent answer wins. The vote then strengthens a repeated mistake instead of correcting it [10]. Sycophantic consensus describes the case where an agent copies a wrong position or gives up a correct one simply because the wrong position has more support [3]. Confidence can come from the agent itself or from its token distribution, but neither source verifies the underlying claim [6]. DebUnc makes this point by comparing deployable uncertainty measures against a diagnostic Ground Truth measure that already knows the answer [6]. The framework instead builds trust from retrieved scientific evidence.

2.2 Literature Review

The review sorts the literature into five areas: debate construction, external verification, sycophancy, uncertainty, and debate evaluation. The main comparison sits between methods that decide when to debate and methods that change agent influence using retrieved evidence. Surveys of LLM-based multi-agent systems map the design space across workflow, infrastructure, and open challenges, and they note that workflow choices affect how errors spread between agents [15]. Recent reasoning-oriented LLMs show that better reasoning needs more than a bigger model. Research has moved toward structured reasoning, multi-agent collaboration, and external verification to make generated responses more reliable, with verification now seen as important as scale [14]. Multi-agent debate is among the most studied of these directions, because agents can challenge each other's reasoning before a final answer is produced, yet it still suffers from conformity and sycophancy when votes are counted [1]. The review below examines representative studies in each area and highlights what each leaves open for trust that is grounded in retrieved evidence.

2.2.1 Similar Applications

The system is not a conventional web or mobile application. It is an experimental deliberation layer that can be integrated into scientific question-answering systems, research assistants, evidence-based decision-support tools, or other applications that require reliable reasoning. Rather than relying on a single model response, these systems generate multiple candidate solutions and aggregate the results after the candidates interact before returning a final answer to the user. Du et al. showed structured debate among multiple agents can improve factuality and reasoning compared with a single-agent approach [1]. Wang et al. showed combining outputs of several models through layered aggregation can improve overall capability without explicit debate [7]. Existing applications differ mainly in how they coordinate agents and combine outputs. Structured debate is one approach. It encourages reasoning refinement between agents [1]. Hierarchical aggregation is another. Later models in a pipeline refine or combine earlier responses [7]. Zhou et al. described a self-organizing setup where the task itself decides which reasoning agents take part, instead of a fixed workflow [16]. Kushwaha and Madhavan built a retrieval-augmented system that spots contradictions between retrieved documents and ranks evidence by source credibility before producing a final response [12]. Both directions show spreading reasoning across specialized agents helps factual consistency and reduces single-agent errors. Dialogue design and scale also matter. Ueda et al. studied how multi-agent LLM dialogues should be structured for research ideation and found dialogue structure affects idea quality [17]. Wang et al. looked at how well LLM-based multi-agent systems scale for scientific code generation and found adding more agents does not always help without better coordination [18]. That aggregation stays fixed once the discussion starts, so a correct minority can still be outvoted. The proposed framework keeps the same goal, reliable answers, but lets agent influence change during the discussion. Evidence verification results are recorded

after every round, and each agent's trust score is updated from claim-level scientific evidence. A well-supported minority opinion can then survive, and an unsupported majority consensus carries less weight.

2.2.2 Related Research

Chain-of-thought prompting improves reasoning in large language models, as Wei et al. (2022) showed [2]. Du et al. (2024) built on this with multi-agent debate. Agents answer a question and revise their positions after looking at peer reasoning [1]. CRITIC uses a different path [13]. It checks a model's response with external tools, so claims can be verified, but only one model works at a time. Mixture-of-Agents is layered, not conversational [7]. Several proposer models write answers and a later stage combines them. The method shows what multiple LLMs can do, yet the combination is static and carries no evidence-based trust. iMAD focuses on cost, not correctness [5]. Fan, Yoon, and Ji extract 41 features from a structured self-critique and let a classifier decide whether to start a debate. They report token savings up to 92 percent against GroupDebate and accuracy gains up to 13.5 percentage points over a single-agent baseline. Once the debate starts, agent influence does not change. Pitre, Ramakrishnan, and Wang studied sycophancy inside debate [3]. In their tests the correct answer was present but ignored in more than 20 percent of wrong-answer cases. CONSENSAGENT in that work detects stalling and answer copying and rewrites the task prompt before the next stage. Measured sycophancy drops, but the final step still leans on self-reported confidence and consistency. DebUnc shares uncertainty between agents instead [6]. Uncertainty reaches peers through prompts or attention scaling, yet deployable measures give only small gains. A diagnostic Ground Truth signal that knows the correct answer gives a much larger gain. The gap between the two shows the real bottleneck is the quality of the trust signal. Kushwaha and Madhavan built an eight-agent MAS-RAG system that finds contradictions among retrieved documents and ranks evidence by credibility [12]. Source credibility there is set by a human curator and never updated during deliberation. This keeps a strong source strong even when its evidence is weak for the current claim, which can keep a wrong majority in place. Other studies look at the social side of debate. Cau et al. found selective persuasion can steer opinion convergence [19]. Yeow et al. compared decision architectures and found feedback between agents can spread early errors [20]. A single LLM with strong in-context examples can sometimes match multi-agent discussion, as Wang et al. showed, and they list judge errors and peer conformity as weak points [10]. Choi, Zhu, and Li separated debate communication from voting. They argue majority voting explains much of the usual gain [9]. FREE-MAD drops final consensus altogether and uses anti-conformity prompts with trajectory scoring [21]. ReSo selects agents through reward-driven dynamic task graphs [16]. These works tune coordination and agent selection, but none checks scientific claims to change agent influence. Coordination helps task allocation, but it does not fix sycophantic conformity at the final vote. Sycophancy also appears at model level. Ibrahim

et al. found tuning models to sound warm and agreeable lowers factual accuracy and raises sycophancy [4]. Chen et al. tackled the same issue with self-augmented preference alignment [22]. Pitre et al. proposed process-level diagnostics for engagement, responsiveness, influence asymmetry, balance, stability, and agent utility. These help when ground truth is not available [23]. Their work shows final agreement alone is not enough to judge a debate, and while metrics like balance and stability help spot poor debates without answers, they do not change trust during debate, so the final step still uses votes. Across all these studies, no method updates an agent's influence continuously from claim-level support in retrieved scientific evidence. A bounded trust score is how this framework closes that gap.

2.3 Gap Analysis

Two gaps already have partial answers. Cost was the first. iMAD predicts whether a debate is worth running before it starts, so extra compute is spent only where it helps [5]. The next gap is unsupported agreement, where conformity or sycophancy pushes a group toward a shared wrong answer. CONSENSAGENT handles this with prompt optimization while the debate runs [3]. The remaining gap is harder. It is deciding which agents deserve more influence when disagreement remains. DebUnc uses uncertainty-based weighting for this, but its trust signal is the model's own internal uncertainty and nothing outside the model verifies it [6]. Agreement is not the same as correctness. Selective persuasion can steer where a group's opinion settles [19], and peer conformity remains a known limit of debate [10]. Retrieval-based systems verify the information they pull in and some also improve the aggregation step. None of them update an agent's influence from verified scientific evidence during the debate [13]. Dynamic coordination methods only choose which agents take part, and trust stays untouched [16]. The gap is a mechanism that updates agent influence from evidence while a debate is still running. The framework does this in steps. An agent response is split into factual claims, relevant scientific passages are retrieved for each claim, each claim is classed as supported, contradicted, or unverifiable, and those verdicts update a bounded trust score after each round. A minority with good support keeps its standing, and a majority without support loses ground. Evaluation reports four rates together: answer accuracy, consensus-collapse, minority-preservation, and evidence-calibration. Retrieval can be sparse and claim decomposition is imperfect. Reasoning errors can also correlate between agents. These limits carry over into the design.

2.4 Summary

The chapter introduced LLM reasoning, chain-of-thought prompting, multi-agent debate, and sycophantic consensus. The reviewed studies cover debate efficiency, prompt correction, uncertainty weighting, external verification, process evaluation, and agent coordination. They do not continuously update agent influence from claim-level scientific evidence. The next chapter uses this gap to define the system requirements, detailed methodology,

system design, and project plan.

Chapter 3

Project Design

Chapter 3 defines the system requirements and lays out the methodology, the design decisions, and the project plan with the task allocation. The first section records the functional and nonfunctional requirements together with the context diagram, the level-one data flow diagram, and the interface design.

3.1 Requirement Analysis

Requirement analysis fixes what the framework must do before any code is written. The system is an experimental deliberation layer rather than a conventional application, so the requirements are framed around the research pipeline. A question enters, agents debate it over a fixed number of rounds, the claims are checked against external evidence, and a trust-weighted answer comes out. Multi-agent debate was chosen because it has been shown to improve factuality over single-model answers [1]. The trust mechanism exists because a confident wrong majority can push a correct minority agent into abandoning its answer [3]. The subsections below define the requirements, the system boundary, the data flows, and the interface through which the framework is operated.

3.1.1 Functional and Nonfunctional Requirements

The functional requirements spell out the services the framework must provide. Questions reach it from the user or from the evaluation harness, and the ground truth stays hidden from every agent. Not every question needs a debate. A confidence gate sorts them first, so the easy ones get a direct answer and the rest go into the pipeline. Orchestration runs three heterogeneous LLM agents through at most three revision rounds, sequenced by a state machine with a retry cap of three.

Each agent's reasoning is then split into atomic factual claims, and every claim is tagged to the agent that made it. A fallback extraction pass covers the cases where structured tagging fails. Retrieval gives each agent its own database: PubMed for the first, arXiv for the second, Semantic Scholar for the third. OpenAlex stands in when a source is down,

and a cross-encoder reranks the passages that come back. Each claim then lands on one of four verdicts: supported, contradicted, unverifiable, or contested.

After every round the trust score is updated as $S_i(t+1) = S_i(t) + \alpha V_i - \beta H_i$. A softmax follows, the values are clamped between 0.1 and 0.9, and the vector is renormalized.

Those two bounds matter. Without them an agent could be silenced outright, or could grow to dominate the debate. An injection point lets the harness insert a fabricated wrong consensus between rounds. That is how sycophantic collapse gets measured under controlled pressure. At the end, weighted aggregation over trust and position picks the final answer, and the majority vote plays no part in it. The result package carries the answer, the citations behind it, the full trust trajectory, and every agent's reasoning trace.

The evaluation harness computes answer accuracy together with consensus-collapse rate, minority-preservation rate, and evidence-calibration rate. It compares the framework against majority voting and against the MoA [7], iMAD [5], and DebUnc [6] baselines. Recovery paths cover the failure modes. Unparseable model output goes through the fallback extraction pass, and a timeout is retried once before the round is marked inconclusive. A downed retrieval API falls back to cached evidence or to OpenAlex.

The nonfunctional requirements describe the qualities of the system rather than its functions. Latency is the first constraint. The debate must respect the round cap of three and the retry cap of three, so latency stays bounded on the target hardware. A failure in one component does not block the whole debate. Every failure path degrades locally and keeps the pipeline moving. Portability matters too. Closed APIs and locally served models both work through the OpenAI-compatible vLLM interface. The pipeline runs unchanged on the development models (Qwen3.5-9B, Gemma 4 12B, Ministral-3-14B) and on the final models (Qwen3.6-27B, Gemma 4 26B, Mistral Small 3.2 24B). Inference fits within a single RTX PRO 6000 Blackwell 96GB rental at about USD 0.70 to 1.90 per hour, with no fine-tuning. Every run is seeded and every intermediate result is logged, so any experiment can be repeated and checked. The output records which agent was supported or contradicted by the evidence, so the decision trail can be audited. The ground truth stays hidden from the agents, and the final output cites the evidence behind the answer.

3.1.2 Context Diagram

The context diagram in Figure 3.1 shows the system boundary and the external actors it exchanges data with. The framework sits in the middle of the diagram. Questions arrive from the user or the evaluation harness. Scientific passages come back from three external databases, and off-the-shelf language models are called through the vLLM server on a rented RTX PRO 6000 Blackwell GPU. The evaluation harness receives the scores, and the user gets the final result package with the answer, the citations, the trust trajectory, and each reasoning trace.

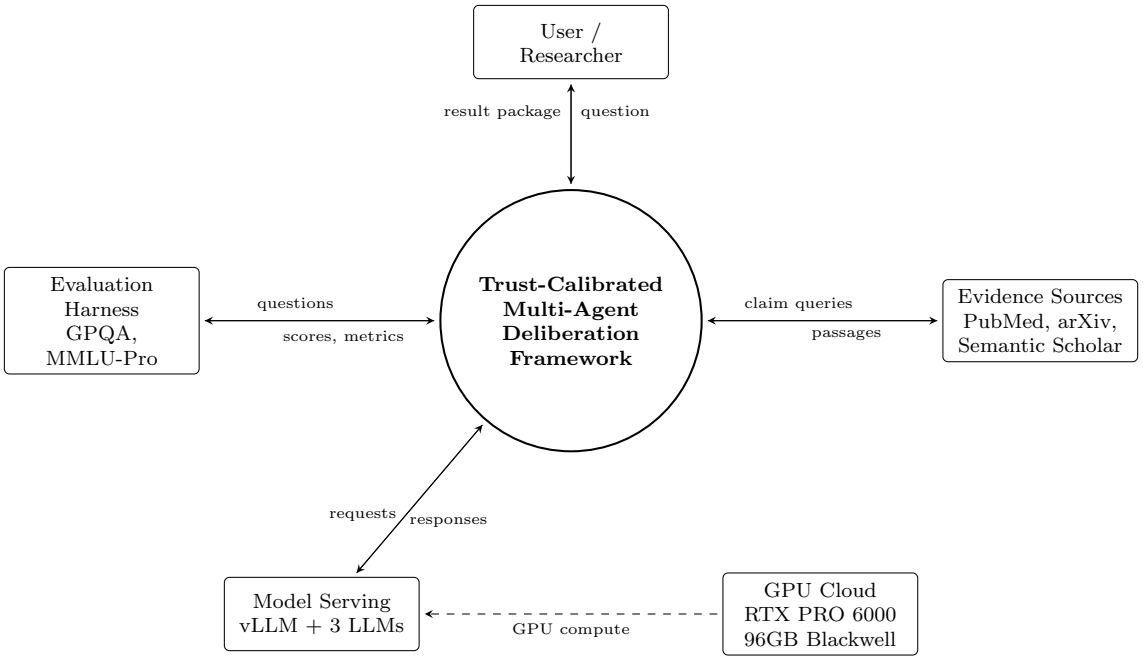

Figure 3.1: Context diagram of the trust-calibrated multi-agent deliberation framework.

3.1.3 Data Flow Diagram Level 1

The level-one data flow diagram in Figure 3.2 decomposes the framework into seven processes. A question enters through the intake and gate process, which decides between a direct answer and a debate. Orchestration runs the revision rounds. It sends inference requests to the external models and reads and writes the debate state. Each position is decomposed into atomic claims and verified against the external sources. The verdicts then drive the trust update, which writes the new trust values back into the debate state. At the last round, weighted aggregation selects the answer and the packaging process logs the result before it returns to the user. Three data stores hold the debate state, the evidence and verdict log, and the results log.

The diagram reads from left to right and top to bottom. The evaluation harness submits a question, and the confidence gate decides whether the debate is worth running. The orchestration process then asks the model server for the agent positions. Positions are decomposed into atomic claims, and the retrieval process verifies them against the external sources. The verdicts are stored in the evidence log before they drive the trust update. The updated trust is written back into the debate state, so the next round starts from the corrected influence values. At the last round the aggregation process selects the answer. The packaging process logs it in the results log, and the result package with the answer and the citations returns to the user.

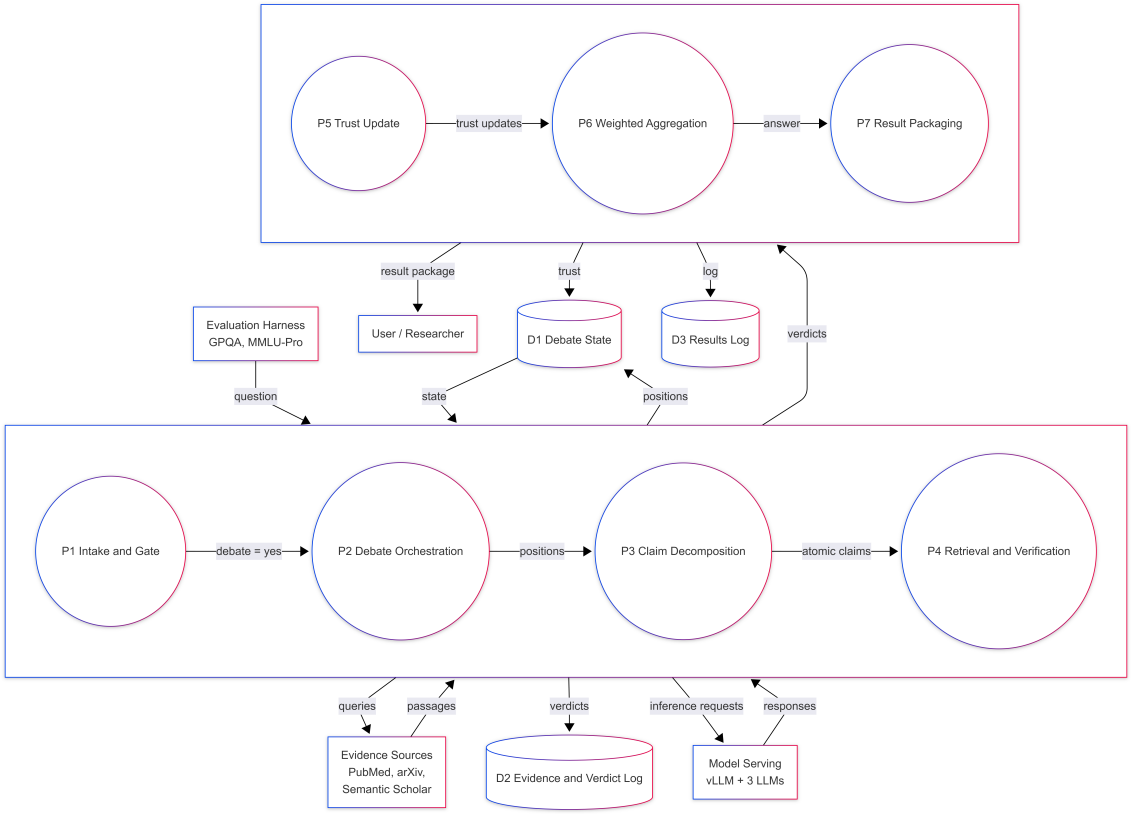

Figure 3.2: Level-one data flow diagram of the trust-calibrated multi-agent deliberation framework.

3.1.4 UI Design

The framework is operated through a command-line interface and a small web dashboard. The command-line interface starts experiments and points the pipeline at the evaluation datasets. Every run is written to a JSON package. It is the primary interface because the system runs in batches on a rented GPU, where a graphical interface adds little value. The web dashboard is used afterwards to inspect single debates. It shows the agents as cards with their current positions, plus a chart of the trust trajectory across rounds and the verdict of every claim with the passage that supports or contradicts it. This is where the transparency of the framework becomes visible, because a user can trace exactly why an agent lost influence. Figure 3.3 and Figure 3.4 show the intended layout of both interfaces. The screenshots will be replaced with the real output of the system once the implementation is complete.

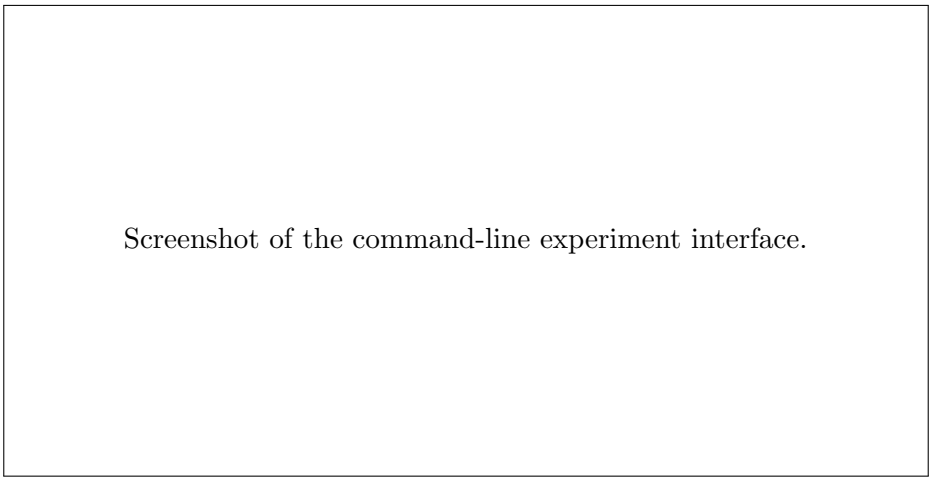

Figure 3.3: Command-line interface for running experiments and writing JSON result packages.

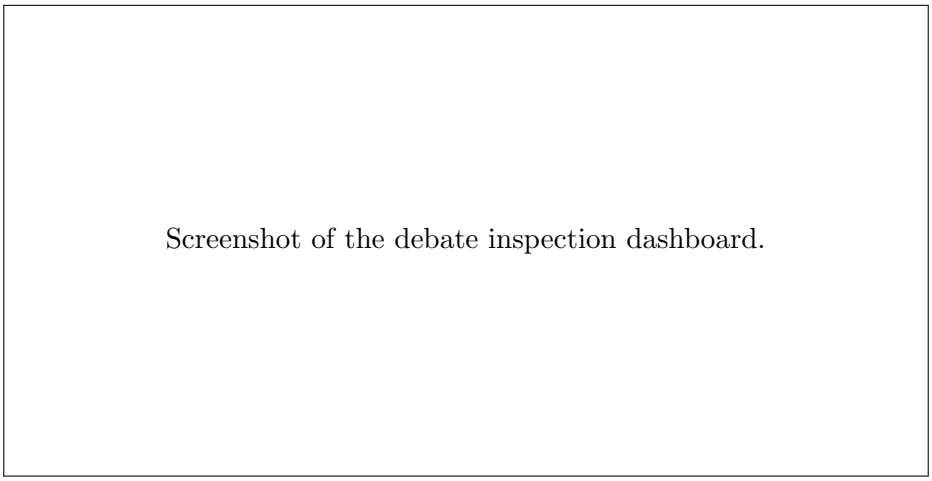

Figure 3.4: Web dashboard showing agent positions, the trust trajectory, and per-claim evidence verdicts.

3.2 Detailed Methodology and Design

3.3 Project Plan

The project runs for ten months, from Month 1 to Month 10, and covers 40 weeks. It is split into eight phases with six review gates. The work starts with literature and setup and ends with writing and submission. Table 3.1 lists each phase with its weeks and deliverables. Table 3.2 lists the gates and their checkpoints. Table 3.3 adds a SWOT view of the plan. This report covers work through the mid-project review in Month 5, while later phases carry the project to the final defence.

The plan starts with scoping. Project scoping in Phase-0 fixes the research question, the trust-calibrated idea, the evaluation shape, and the gate criteria. Data and model work runs in parallel. Datasets GPQA and MMLU-Pro are checked, and the three development models are chosen with their serving setup. Analysis of the baseline follows in Phase-1.

Injection tests and baselines B1 to B4 are run, and the pilot checks if a wrong majority can still fool voting. Design and build fill Phase-2. Claim decomposition and source-partitioned RAG are built, and the trust function is tuned with its clamp and softmax. A short dry run and design freeze close Phase-2 to 3. The main study sits in Phase-3a and Phase-3b. Phase-3a runs core conditions with three seeds, and Phase-3b runs ablations with different agent counts and alpha/beta values. Human study and failure review occupy Phase-4. Writing, reproducibility packing, submission, and defence preparation close the project in Phase-5.

Table 3.1: Project phases with week-level durations and deliverables.

Phase	Duration	Deliverables
Phase-0: Literature and Setup	Month 1 (Wk 1–4)	Literature freeze, vLLM and LangGraph setup, model identification, vanilla MAD reproduction
Phase-1: Injection and Baselines	Month 2 (Wk 5–8)	Injection protocol, baselines B1–B4, Proposition 1 proof, month-end pilot
Phase-2: Trust Mechanism Build	Months 3–4 (Wk 9–16)	Claim decomposition, source-partitioned RAG, trust function v1, baselines B5/B6/B9
Phase-2 to 3: Mid-Project	Month 5 (Wk 17–20)	Design freeze, dry run on one dataset, FYDP-1 defence preparation, and review feedback fixes
Phase-3a: Main Experiments	Month 6 (Wk 21–24)	Core conditions on primary datasets, three seeds, 95% confidence intervals, and effect size report
Phase-3b: Ablations and Scaling	Month 7 (Wk 25–28)	Four ablations, α/β sweep, $N \in \{2, 3, 5\}$, and results freeze
Phase-4: Human Evaluation and Analysis	Month 8 (Wk 29–32)	Human evaluation with $n = 60$, ECR calibration, failure analysis, and buffer for slippage
Phase-5: Writing and Submission	Months 9–10 (Wk 33–40)	Thesis and paper drafting, reproducibility package, submission, and FYDP-2 defence preparation

The phase order follows dependencies. Vanilla MAD reproduction in Phase-0 must finish before injection validation in Phase-1 can be trusted. The trust mechanism in Phase-2 needs the injection harness from Phase-1, so it cannot start early. Main experiments in

Table 3.2: Review gates and their checkpoints.

Gate	Due	Checkpoint
Gate 0	End of Month 1	Vanilla MAD reproduces the Du et al. baseline on a GPQA slice, and the reference list is frozen
Gate 1 (Go/No-Go)	End of Month 2	Injection validation reaches $\kappa \geq 0.75$, baseline CCR sits at 0.30 or higher, pilot executed
Gate 2	End of Month 4	Trust mechanism plus source-partitioned RAG plus competitor baselines, first CCR and MPR on one dataset
FYDP-1 Defence	Month 5	Mid-project report with design freeze and initial results
Gate 3	End of Month 7	Full results freeze with confidence intervals and effect sizes
Submit and FYDP-2	Months 9–10	Paper submitted, thesis completed, final defence

Phase-3a wait for the trust function and the dry run in Phase-2 to 3. Human evaluation in Phase-4 needs the frozen results from Phase-3b. Writing in Phase-5 overlaps only with final checks, not with core experiments.

Two buffers guard the critical path. A dry run on one dataset sits inside Month 5 before the defence, and Month 8 keeps a full month free for slippage. Both buffers sit where a late failure would otherwise hit a fixed deadline. Weekly meetings and Git version control track progress, and every gate review decides Go or Fix before the next phase starts.

Phase-0 covers Week 1 to Week 4. Week 1 and Week 2 focus on literature freeze and reference list lock, with every paper checked against its DOI. Week 3 sets up vLLM and LangGraph, and Week 4 tests the model trio on a small GPQA slice. The vanilla MAD baseline must reproduce Du et al. results on that slice before Phase-1 starts.

Phase-1 runs Week 5 to Week 8. Week 5 builds the injection harness that inserts a fabricated wrong consensus between rounds. Week 6 to Week 8 validate the injection with kappa checks and run baselines B1 to B4. The month ends with a pilot that must reach CCR 0.30 or higher to pass Gate 1. Phase-2 spans Week 9 to Week 16. Week 9 to Week 12 build claim decomposition. Source-partitioned RAG with cross-encoder reranking is added in Week 13 to Week 16.

Phase-2 to Phase-3 covers Week 17 to Week 20. This window freezes the design and runs a dry experiment on one dataset for the mid-project review. Phase-3a runs Week 21 to Week 24 with core conditions on primary datasets using three seeds and 95 percent intervals. Phase-3b runs Week 25 to Week 28 with four ablations and the N and alpha/beta

Table 3.3: SWOT analysis for the project plan.

Strengths	**Weaknesses**
Clear gap on sycophantic consensus, bounded trust design with no fine-tuning, six-member team with mixed skills, and expert supervision that keeps scope tight. The design needs no new training and runs on a single rented GPU.	Sparse retrieval on rare claims, imperfect claim split, correlated reasoning errors between agents, and limited GPU budget for 300 hours. Small sample noise in early pilots can also hide real gains.
Opportunities	**Threats**
Growing interest in multi-agent debate and sycophancy, open datasets GPQA and MMLU-Pro, free retrieval APIs from PubMed, arXiv, Semantic Scholar and OpenAlex, and affordable cloud GPU rental with Blackwell FP8 support.	API rate limits and downtime, model price changes, time slip from complex integration, and evaluation noise on small samples. A weak retrieval week can shift trust scores and hide the true effect.

sweeps. Phase-4 holds human evaluation with $n = 60$ in Week 29 to Week 32. Phase-5 uses Week 33 to Week 40 for thesis and paper writing with a reproducibility package.

Resource use is heaviest in Phase-3. The 300 GPU hours are split with about 80 hours for Phase-3a and Phase-3b each, and the rest for earlier phases. A retrieval API failure falls back to cached evidence or OpenAlex, so the debate still runs. A dry run failure at Gate 2 is absorbed by the buffer in Month 8 without moving the final defence.

3.4 Task Allocation

Six members split the work along the modules of the framework. Table 3.4 fixes who owns which primary modules, and Figure 3.5 shows when each member is active across the ten months.

Table 3.4: Module ownership across the six team members.

Member	Primary modules
Md. Atikur Rahaman (Leader)	Architecture, trust mechanism, coordination
Rakibul Hasan	Vanilla MAD reproduction, injection protocol
Md. Salman Rohoman Nayeem	Claim decomposition, source-partitioned RAG
Pratay Paul	Evaluation harness, CCR/MPR/ECR metrics
Yousuf Kamal Himel	Baselines B1–B9, vLLM serving
Mst. Farjana Akter Limu	Evidence APIs, dashboard, result packages

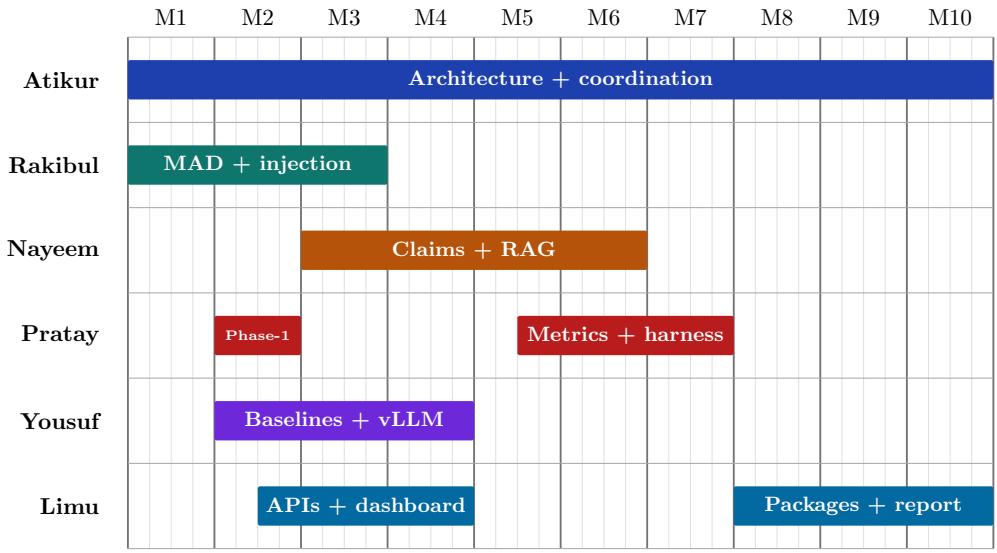

Figure 3.5: Gantt view of member activity across the ten project months, following the phase plan in Table 3.1.

Each task has a clear owner and a short deliverable. The owner pairs with the phase in Table 3.1, so every phase has at least one accountable member. Dependencies are handled by order. Vanilla reproduction finishes before injection starts, and the injection harness finishes before the trust build starts. The dry run must pass before main experiments start and human evaluation waits for the results freeze. Writing uses the frozen results and runs last. The leader reviews each module before it is marked complete. Weekly Git commits and short stand-ups track slippage, and any missed gate triggers a fix week before the next phase begins.

3.5 Summary

Chapter 3 fixed the functional and nonfunctional requirements and drew the system boundary in the context diagram. The data flow diagram and the interface design follow. The design section compared the main alternatives and recorded why each choice was made. The project plan then locks eight phases and six gates that carry the work to the final defence. Implementation and evaluation come next.

Chapter 4

Implementation and Results

This chapter records the environment the framework runs in and the way it is tested. Full results belong to the final report, so this chapter keeps the setup and the evaluation plan.

4.1 Environment Setup

4.2 Testing and Evaluation

4.3 Results and Discussion

4.4 Summary

Chapter 5

Standards and Design Constraints

This chapter records the standards the framework follows, the constraints that shaped its design, and the budget behind the experiments. It also maps the project against the complex engineering problem criteria and closes with a short summary.

5.1 Compliance with the Standards

Standards matter in three places: the software stack, the hardware the framework runs on, and the communication between components. Each area had alternatives, and each choice is justified below.

5.1.1 Software Standards

Python 3.11 was chosen as the implementation language. The LLM ecosystem is built around it, so vLLM, the Hugging Face libraries, and the evaluation tooling all ship Python support first. Java and C++ are faster in raw terms, but neither has the same depth of ready-made support for model serving and retrieval work. The framework is a research prototype, not a latency-critical production service, so ecosystem fit outweighs raw speed. Surveys of LLM-based multi-agent systems describe this design space in terms of workflow, infrastructure, and open challenges [15], and the choices below follow that map.

PyTorch 2.x is the deep learning framework underneath vLLM. It is the de facto standard for LLM inference and supports the Qwen, Gemma, and Mistral families directly. TensorFlow and JAX were considered. TensorFlow's LLM serving story is weaker, and JAX would pull the whole pipeline onto its own stack without any benefit here.

vLLM serves the models through an OpenAI-compatible API. That interface matters because the framework must work with closed APIs and locally served models without touching the orchestration code. PagedAttention gives vLLM high throughput under the three-agent debate load, and its FP8 support matches the Blackwell GPU. Ollama is easier to set up but slower under concurrent requests. llama.cpp runs well on CPU but gives up the GPU throughput the debate needs. TGI is a close alternative, and vLLM won on ecosystem fit and the OpenAI-compatible surface.

Sentence-transformers provides the cross-encoder that reranks retrieved passages. BM25 was the fallback, but it matches on surface terms only, while the cross-encoder scores semantic fit between a claim and a passage. The web dashboard is served with FastAPI, and every result package is written as JSON following RFC 8259, validated against a JSON Schema. Git tracks the code and LaTeX (Overleaf) produces this report.

5.1.2 Hardware Standards

The experiments run on a single NVIDIA RTX PRO 6000 Blackwell GPU with 96 GB of GDDR7 memory. The choice came down to memory capacity and architecture. The development models (Qwen3.5-9B, Gemma 4 12B, Ministral-3-14B) fit comfortably in BF16. The final models (Qwen3.6-27B, Gemma 4 26B, Mistral Small 3.2 24B) total about 77 billion parameters, which no single card holds in FP16, so they run in FP8. Blackwell has native FP8 support, and that is exactly the point of difference.

The A100 80GB was the first alternative considered. It has enough memory for the development models and strong HBM bandwidth, but its FP8 path is weaker and the architecture is two generations old. The H100 80GB is faster yet rents for roughly twice the price and brings nothing the framework needs. The L40S at 48 GB and the consumer RTX 5090 at 32 GB cannot hold the final model trio at all. The RTX PRO 6000 sits at the right point: enough memory, modern quantization, and a rental price that fits a student budget. The host machine needs at least 64 GB of system RAM and NVMe storage to stage the models before they move to the GPU.

5.1.3 Communication Standards

All communication between the framework and its components runs over HTTP with TLS. The model server exposes the OpenAI-compatible chat completions endpoint, which is REST with JSON payloads. gRPC was the alternative for model serving. It is faster in benchmark terms, but the OpenAI-compatible surface keeps the framework portable between closed APIs and local vLLM servers, and that portability outweighs the latency difference.

The evidence sources are reached through their public REST interfaces: PubMed E-utilities, the arXiv API, the Semantic Scholar Graph API, and OpenAlex as the fallback. Each returns JSON or XML over HTTPS, and the framework handles rate limits with the retry caps defined in the requirements. WebSocket is reserved for the dashboard's live trust chart. Nothing else needs a persistent connection.

5.2 Design Constraints

5.2.1 Economic Constraint

5.2.2 Environmental Constraint

5.2.3 Ethical Constraint

5.2.4 Health and Safety Constraint

5.2.5 Social Constraint

5.2.6 Political Constraint

5.2.7 Sustainability

5.3 Cost Analysis

The budget covers the rented GPU, storage, and the free research services. No fine-tuning is needed, so cost is mostly hours on a rented card. The largest line is inference time on the RTX PRO 6000 Blackwell, estimated at about 300 hours across development and evaluation. The hours are not spread evenly. Phase-0 and Phase-1 use about 30 hours for setup and baselines. Phase-2 uses about 60 hours for building claim split and RAG. Phase-3a and Phase-3b together use about 140 hours for main runs and ablations. Phase-4 uses about 40 hours for human study checks, and Phase-5 uses about 30 hours for final runs and packaging. This split shows why Phase-3 dominates cost. Table 5.1 summarizes the primary budget, and Table 5.2 breaks the GPU hours by phase.

Table 5.1: Primary budget for the project.

Item	Quantity	Unit Cost (USD)	Total (USD)
RTX PRO 6000 Blackwell 96GB rental	300 hours	0.70–1.90 / hour	210–570
Retrieval APIs (PubMed, arXiv, S2, OpenAlex)	—	0 (free tier)	0
Datasets (GPQA, MMLU-Pro)	—	0 (public)	0
Storage (model weights, logs)	200 GB	0.05 / GB	10
Total			**220–580**

Table 5.2: GPU hours and cost split by phase (RTX PRO 6000, USD 0.70–1.90 per hour).

Phase	Hours	Cost (USD)
Phase-0 and Phase-1 (setup, baselines)	30	21–57
Phase-2 (trust build)	60	42–114
Phase-3a and Phase-3b (main runs)	140	98–266
Phase-4 (human eval)	40	28–76
Phase-5 (writing, final checks)	30	21–57
Total	**300**	**210–570**

Storage is 200 GB at USD 0.05 per GB, about USD 10. Model weights with logs and result packages take about 180 GB, and workspace takes the rest. Retrieval APIs PubMed, arXiv, Semantic Scholar and OpenAlex run on free tiers with rate limits, so the direct cost is zero, but limits add time. Datasets GPQA and MMLU-Pro are public, so the cost is also zero. Printing and internet take a small fixed amount, about USD 15, and Overleaf is covered by the team. No extra license is needed.

An alternate budget was also prepared. Renting an A100 80GB instead costs about USD 0.68 to 1.50 per hour, or 204–450 for the same 300 hours. The saving is real but small, and it comes at a cost: weaker FP8 support means the final model trio would need tighter quantization or sequential loading, which adds engineering time and risk. The RTX PRO 6000 was selected because the Blackwell FP8 path keeps the pipeline simple and avoids that extra work.

The project has no revenue model. It is an academic deliverable, funded by the university and the team, and its outputs are the report and the open-source framework. Table 5.3 shows the alternate budget for comparison.

Table 5.3: Alternate budget using the A100 80GB.

Item	Quantity	Unit Cost (USD)	Total (USD)
A100 80GB rental	300 hours	0.68–1.50 / hour	204–450
Retrieval APIs	—	0 (free tier)	0
Datasets	—	0 (public)	0
Storage	200 GB	0.05 / GB	10
Total			**214–460**

A 10 percent buffer, about USD 25 to 55, is kept for price shifts and extra hours for failed runs. If a run fails or a gate needs a fix week, the buffer covers up to 30 extra hours without changing the final plan. Cost control is simple. Runs are batched, logs are reused for ablations, and caching avoids repeat retrieval calls.

5.4 Complex Engineering Problem

5.4.1 Complex Problem Solving

The project qualifies as a complex engineering problem under the Washington Accord criteria. Table 5.4 maps the seven attributes P1–P7, and the subsections that follow give the rationale for each mapping.

Table 5.4: Mapping with complex problem solving (Washington Accord P1–P7).

P1 Depth of Knowledge	P2 Range of Conflicting Requirements	P3 Depth of Analysis	P4 Familiarity of Issues	P5 Extent of Applicable Codes	P6 Extent of Stakeholder Involvement	P7 Interdependence
√	√	√	√	√	×	√

P1: Depth of Knowledge Required

The framework spans machine learning, natural language processing, retrieval-augmented generation, and distributed model serving. Solving it required specialist knowledge in these areas, from the trust update formula to the vLLM serving layer. Coverage for P1 is judged per Washington Accord K1–K8, as detailed in Table 5.5 after P7.

P2: Range of Conflicting Requirements

P2 is covered. The project had to balance several needs that pull in different directions. Accuracy competes with latency and cost. More rounds improve the signal but raise inference time and the GPU bill. Trust calibration competes with diversity, because strong weighting can silence a minority that the evidence has not yet covered. Evidence coverage competes with API rate limits. The team had to fix these together. Round caps and clamping bounds were set, and verdicts are handled in a conservative way, so no single need dominates.

P3: Depth of Analysis

P3 is covered. The problem has no textbook solution. The team had to run a comparative experiment with several baselines: majority voting, MoA, iMAD, and DebUnc. Metrics were built for the failure mode. The set includes four rates: answer accuracy, consensus-collapse, minority-preservation, and evidence-calibration. The injection point for a fabricated wrong consensus was built so the collapse can be measured under controlled pressure. This depth of test is why P3 is marked as covered.

P4: Familiarity of Issues

P4 is covered. Sycophantic consensus in multi-agent LLM debate is a new research area. The closest work appeared between 2024 and 2026, and no standard solution exists. The issues were not familiar to the team at the start. A full literature review was done first, as shown in Chapter 2, before any design was fixed. This unfamiliarity is why P4 is marked as covered.

P5: Extent of Applicable Codes

P5 is covered. General software standards apply. JSON (RFC 8259), HTTP over TLS, the OpenAI-compatible API, and JSON Schema validation all apply. No standard covers evidence-grounded trust in LLM debate. The area is therefore only partly standardized, and the framework defines its own rules for the trust score and the four verdicts. This mix of existing codes and new conventions is why P5 is marked as covered.

P6: Extent of Stakeholder Involvement

P6 is not covered. The project has no external stakeholders. The only people involved are the supervisor and the examiners. There is no industry partner and no community group that sets needs or reviews the result during the work. Future users of scientific QA tools could benefit, but they did not take part in design or testing. The team built the system as an academic FYDP, not as a product for a real client. The involvement is therefore narrow and limited to the university. For this reason P6 is marked as not covered, while the other P points remain covered.

P7: Interdependence

P7 is covered. The confidence gate, the orchestrator, the claim decomposer, the retrieval subsystem, the trust updater, and the aggregator are tightly linked. A failure in retrieval cascades into the verdicts and then into the final answer through the trust score. The failure-isolation rule in Section 3.1.1 was added because of this link. The high coupling is why P7 is marked as covered.

Table 5.5 details the profile. The paragraphs below explain each K for this project and why it is covered or not.

Table 5.5: Knowledge profile mapping for P1 (K1–K8).

Knowledge	Applied	Where used
K1: Natural sciences	×	Not directly used; project is computing, not physics or chemistry
K2: Mathematics	√	Softmax, clamping, renormalization, probability
K3: Engineering fundamentals	√	System design, state machines, failure handling
K4: Specialist knowledge	√	LLMs, multi-agent debate, RAG, sycophancy
K5: Engineering methods	√	Experimental methodology, baseline comparison
K6: Computational methods	√	Python, PyTorch, vLLM, FastAPI
K7: Codes and practices	√	JSON, HTTP, TLS, version control
K8: Research and context	√	Ethical constraints and use in scientific QA

K1: Natural sciences K1 is not covered. The project does not use physics or chemistry, and it does not use biology. It is a computing project that works with language models and retrieval. Natural science theory is not applied, so K1 is marked as not covered.

K2: Mathematics K2 is covered. The trust update uses softmax, clamping, and renormalization, and the metrics use probability and statistics. These maths are applied in the trust formula and in the evaluation.

K3: Engineering fundamentals K3 is covered. System design and failure handling with state machines are used throughout. Requirements and the confidence gate with the orchestration logic follow basic engineering design.

K4: Specialist knowledge K4 is covered. LLMs, multi-agent debate, RAG, and sycophancy are the core of the work. This specialist view shapes the claim split, the retrieval, and the trust score.

K5: Engineering methods K5 is covered. The study uses an experimental method with controlled variables. Baselines are compared and results are reported with confidence intervals. Injection is tested in the same setup.

K6: Computational methods K6 is covered. Python, PyTorch, and vLLM serve the models, and FastAPI serves the dashboard. Sentence-transformers reranks passages, and Git tracks changes.

K7: Codes and practices K7 is covered. JSON under RFC 8259, HTTP with TLS, the OpenAI-compatible API, and JSON Schema are used. Version control and code style follow standard practice.

K8: Research and context K8 is covered. Ethical constraints on AI output and the use of scientific QA provide the context. Literature review and gap analysis set the research direction.

The profile is broad rather than deep in one spot. The report applies K4 and K5 in every chapter, and K6 and K7 in the implementation, which shows the range the problem demands.

5.4.2 Engineering Activities

Table 5.6 maps the project to the five engineering activities A1–A5, with rationales below.

Table 5.6: Mapping with complex engineering activities (A1–A5).

A1	A2	A3	A4	A5
Range of resources	Level of Interaction	Innovation	Consequences for society and environment	Familiarity
√	√	×	×	√

A1: Range of Resources

A1 is covered. The project draws on three model families, four literature APIs, a rented Blackwell GPU, and six team members with different strengths. Coordinating these resources is a clear part of the work. The plan in Chapter 3 assigns each task to a named member and tracks weeks and deliverables. This wide range is why A1 is marked as covered.

A2: Level of Interaction

A2 is covered. The system interacts heavily with the outside world. Every debate round calls three external model endpoints and three literature APIs, and the cloud GPU is reached over the network. The team also interacts through a shared repository and weekly reviews. This high level of interaction is why A2 is marked as covered.

A3: Innovation

A3 is not covered. The framework combines existing ideas: multi-agent debate and RAG with trust weighting and reranking. It does not create a new device or a new theory. The change is a bounded trust score that updates with evidence, which is useful but stays at an incremental level. There is no patent-level or breakthrough innovation. For this reason A3 is marked as not covered, even though the combination itself is new for this project.

A4: Consequences for Society and Environment

A4 is not covered. The work is a lab prototype, not a deployed product. Social impact is limited to more reliable research answers in a test setting, not a change in daily life or a large user group. Environmental impact is also small. The test uses about 300 GPU hours on shared cloud without fine-tuning, so energy use stays low. There is no large-scale rollout, so broad consequences are not present. For this reason A4 is marked as not covered.

A5: Familiarity

A5 is covered. Multi-agent debate and evidence verification were new territory for the team. The literature review phase and the baseline study with the step-by-step implementation plan were all built around that fact. Each module was started only after its background was understood. This low familiarity at the start is why A5 is marked as covered.

5.5 Summary

The chapter fixed the standards behind the software, the hardware, and the communication layers, and recorded the economic, environmental, ethical, and social constraints that shaped the design. The budget analysis shows the project fits a student budget on a rented RTX PRO 6000 Blackwell GPU with a phase-wise split for the 300 hours and a 10 percent buffer. The complex engineering mapping shows six of seven P attributes are covered, with P6 not covered due to limited external stakeholder involvement. For knowledge, K2 to K8 are covered while K1 is not. For activities, A1 and A2 are covered and A5 is also covered, while A3 and A4 are not.

Chapter 6

Conclusion

6.1 Summary

6.2 Limitation

6.3 Future Work

References

[1] Yilun Du, Shuang Li, Antonio Torralba, Joshua B. Tenenbaum, and Igor Mordatch. Improving factuality and reasoning in language models through multiagent debate. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235, pages 11733–11763, 2024.

[2] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H. Chi, Quoc V. Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems*, NIPS '22, Red Hook, NY, USA, 2022. Curran Associates Inc.

[3] Priya Pitre, Naren Ramakrishnan, and Xuan Wang. CONSENSAGENT: Towards efficient and effective consensus in multi-agent LLM interactions through sycophancy mitigation. In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 22112–22133, Vienna, Austria, July 2025. Association for Computational Linguistics.

[4] L. Ibrahim, F. S. Hafner, and L. Rocher. Training language models to be warm can reduce accuracy and increase sycophancy. *Nature*, 652:1159–1165, April 2026.

[5] Wei Fan, JinYi Yoon, and Bo Ji. iMAD: Intelligent multi-agent debate for efficient and accurate LLM inference. In *Proceedings of the Fortieth AAAI Conference on Artificial Intelligence*, pages 29403–29411, 2026.

[6] Luke Yoffe, Alfonso Amayuelas, and William Yang Wang. DebUnc: Improving large language model agent communication with uncertainty metrics. In *Findings of the Association for Computational Linguistics: EMNLP 2025*, pages 23299–23315, Suzhou, China, November 2025. Association for Computational Linguistics.

[7] Junlin Wang, Jue Wang, Ben Athiwaratkun, Ce Zhang, and James Y Zou. Mixture-of-agents enhances large language model capabilities. In Y. Yue, A. Garg, N. Peng, F. Sha, and R. Yu, editors, *International Conference on Learning Representations*, volume 2025, pages 33944–33963, 2025.

[8] Chuan He, Zebin Chen, Zhengyi Yang, Shaobo Qiao, Mingchen Ju, Jiate Liu, Dong Wen, and Guanfeng Liu. Minority sentinel: When to overturn majority voting in

multi-agent LLM debates. In *Proceedings of the AgentSearch Workshop at SIGIR 2026*, Melbourne, Australia, 2026. arXiv:2606.29270.

[9] Hyeong Kyu Choi, Jerry Zhu, and Sharon Li. Debate or vote: Which yields better decisions in multi-agent large language models? In *Advances in Neural Information Processing Systems*, 2025.

[10] Qineng Wang, Zihao Wang, Ying Su, Hanghang Tong, and Yangqiu Song. Rethinking the bounds of LLM reasoning: Are multi-agent discussions the key? In Lun-Wei Ku, Andre Martins, and Vivek Srikumar, editors, *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 6106–6131, Bangkok, Thailand, August 2024. Association for Computational Linguistics.

[11] Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. Dense passage retrieval for open-domain question answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 6769–6781, Online, 2020. Association for Computational Linguistics.

[12] Pranjal Kushwaha and S. Akshat Madhavan. Agent-based contradiction detection and resolution for multi-source retrieval augmented generation systems. *International Journal of Scientific Research in Engineering and Management (IJSREM)*, 10(7), July 2026.

[13] Zhibin Gou, Zhihong Shao, Yeyun Gong, Yelong Shen, Yujiu Yang, Nan Duan, and Weizhu Chen. CRITIC: Large language models can self-correct with tool-interactive critiquing. In *The Twelfth International Conference on Learning Representations*, 2024.

[14] D. Zhang, Z.-Z. Li, M.-L. Zhang, J. Zhang, Z. Liu, Y. Yao, H. Xu, J. Zheng, X. Chen, Y. Zhang, F. Yin, J. Dong, Z. Guo, L. Song, and C.-L. Liu. From system 1 to system 2: a survey of reasoning large language models. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, pages 1–20, 2025.

[15] Xinyi Li, Sai Wang, Siqi Zeng, Yu Wu, and Yi Yang. A survey on LLM-based multi-agent systems: workflow, infrastructure, and challenges. *Vicinagearth*, 1(9), 2024.

[16] Heng Zhou, Hejia Geng, Xiangyuan Xue, Li Kang, Yiran Qin, Zhiyong Wang, Zhenfei Yin, and Lei Bai. Reso: A reward-driven self-organizing llm-based multi-agent system for reasoning tasks. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 15979–15998. Association for Computational Linguistics, 2025.

[17] Keisuke Ueda, Wataru Hirota, Takuto Asakura, Takahiro Omi, Kosuke Takahashi, Kosuke Arima, and Tatsuya Ishigaki. Exploring the design of multi-agent LLM dialogues for research ideation. In *Proceedings of the 26th Annual Meeting of the Special Interest Group on Discourse and Dialogue*, pages 322–337, Avignon, France, 2025. Association for Computational Linguistics.

[18] Yuru Wang, Kaiyan Zhang, Kai Tian, Sihang Zeng, Xingtai Lv, Ning Ding, Biqing Qi, and Bowen Zhou. Scalability of LLM-based multi-agent systems for scientific code generation: A preliminary study. In *Proceedings of The 3rd Workshop on Mathematical Natural Language Processing (MathNLP 2025)*, pages 50–61, Suzhou, China, 2025. Association for Computational Linguistics.

[19] E. Cau, V. Pansanella, D. Pedreschi, et al. Selective agreement, not sycophancy: investigating opinion dynamics in LLM interactions. *EPJ Data Science*, 14:59, August 2025.

[20] Xian Yeow, Shunichi Lee, Lasitha Akatsuka, Vidyaratne Aman, A Hareesh Kumar, Chetan Farahat, and A Gupta. Reliable decision-making for multi-agent LLM systems. *arXiv preprint arXiv:2406.04092*, 2025.

[21] Yu Cui, Hang Fu, Haibin Zhang, Licheng Wang, and Cong Zuo. FREE-MAD: Consensus-free multi-agent debate. In *Findings of the Association for Computational Linguistics: ACL 2026*, 2026.

[22] Chien-Hung Chen, Hen-Hsen Huang, and Hsin-Hsi Chen. Self-augmented preference alignment for sycophancy reduction in LLMs. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 12379–12391, Suzhou, China, 2025. Association for Computational Linguistics.

[23] Priya Pitre, Gaurav Srivastava, Lu Zhang, Le Wang, Naren Ramakrishnan, and Xuan Wang. A diagnostic study of multi-agent llms for real-world debates. In *Proceedings of the 43rd International Conference on Machine Learning*, volume 306 of *PMLR*, Seoul, South Korea, 2026.